PROJET D'ÉCONOMÉTRIE

Économétrie des Séries Temporelles

Rentabilités d'actifs financiers et processus ARMA simulés : stationnarité, autocorrélation et comparaison

Réalisé par : Chaimaa MOUSTAOUI

Établissement : Université Abdelmalek Essaâdi

Outils : R (analyse principale) · Python (automatisation)

Date : Septembre 2026

Introduction

Ce rapport étudie les propriétés statistiques de séries temporelles, en particulier leur **stationnarité** et leur **structure d'autocorrélation**, à partir de deux types de données : dix séries financières réelles téléchargées sur Yahoo Finance et dix processus ARMA stationnaires simulés. L'objectif est de comprendre ce que les modèles AR, MA et ARMA peuvent (ou non) reproduire du comportement des marchés.

La période d'étude s'étend du 1er janvier 2020 au 1er janvier 2025. Elle inclut le krach lié au Covid-19 (mars 2020), la phase de remontée des taux d'intérêt (2022-2023) et la forte hausse des actifs risqués en 2024. L'analyse principale est réalisée avec **R** (packages *quantmod*, *moments* et fonctions *arima.sim* et *acf*) ; **Python** (*yfinance*, *statsmodels*) est utilisé dans la dernière partie pour automatiser l'ensemble de la démarche.

Le rapport suit le plan du guide du projet : (1) données réelles, (2) données simulées, (3) comparaison, (4) automatisation en Python. Les codes complets figurent en annexe.

1. Données réelles

1.a Sélection des séries

Dix séries de prix de clôture quotidiens ont été téléchargées, de façon à couvrir des classes d'actifs variées (actions, indices, obligations, change, matières premières, crypto-monnaie). Le tableau 1 en donne la liste.

Tableau 1 — Séries téléchargées (Yahoo Finance, 01/01/2020 – 01/01/2025)

Actif	Ticker	Type d'actif (rubrique Yahoo Finance)	Nb de prix
S&P 500	**^GSPC**	**Indice boursier (World Indices)**	**1 258**
ETF Techno	**XLK**	**ETF sectoriel — technologie (Sectors / ETF)**	**1 258**
Apple	**AAPL**	**Action (Stocks)**	**1 258**
Microsoft	**MSFT**	**Action (Stocks)**	**1 258**
Obligations (TLT)	**TLT**	**ETF obligataire — Trésor américain long terme (Bonds)**	**1 258**

ETF S&P 500 (SPY)	**SPY**	**ETF répliquant le S&P 500 (ETFs)**	**1 258**
Euro/Dollar	**EURUSD=X**	**Taux de change (Currencies)**	**1 305**
Bitcoin	**BTC-USD**	**Crypto-monnaie (Crypto)**	**1 827**
Futur Pétrole	**CL=F**	**Contrat à terme — pétrole brut WTI (Futures)**	**1 258**
Or (Gold)	**GC=F**	**Contrat à terme — or (Futures)**	**1 258**

Le nombre d'observations diffère selon le calendrier de cotation : les marchés d'actions et de futures ne cotent que les jours ouvrés (≈ 1 258 jours), le change cote 5 jours sur 7 (1 305) et le Bitcoin 7 jours sur 7 (1 827).

1.b Transformation des prix en rentabilités

Les prix P_t sont transformés en rentabilités logarithmiques (sous R : *diff(log(prix))*) :

$$r_t = ln(P_t / P_{t-1}) = ln\, P_t - ln\, P_{t-1}$$

Ce choix est standard en finance : la rentabilité logarithmique approche la variation relative pour de petites variations, elle est additive dans le temps (la rentabilité sur plusieurs jours est la somme des rentabilités journalières) et elle ne dépend pas du niveau du prix, ce qui la rend a priori plus apte à être stationnaire. Les figures 1a et 1b présentent, pour chaque actif, le prix (à gauche) et la rentabilité (à droite).

Point d'attention sur le pétrole. Le 20 avril 2020, le contrat WTI a clôturé à environ –37 $: le logarithme n'est pas défini pour un prix négatif. Le calcul produit donc des valeurs manquantes (*NaN*) — c'est l'origine de l'avertissement *invalid value encountered in log* rencontré en Python — qui sont supprimées (*na.omit* en R, *dropna* en Python). Deux rentabilités (l'entrée et la sortie de ce prix négatif) sont ainsi perdues, ce qui est négligeable sur ≈ 1 250 observations mais doit être signalé.

Figure 1a — Prix (bleu) et rentabilités logarithmiques (orange) : S&P 500, ETF Techno, Apple, Microsoft, TLT. Source : calculs de l'auteur (R).

Figure 1b — Prix et rentabilités : SPY, Euro/Dollar, Bitcoin, pétrole (WTI), or. Source : calculs de l'auteur (R).

1.c Statistiques descriptives des rentabilités

Tableau 2 — Statistiques descriptives des rentabilités journalières (R)

Actif	Moyenne	Écart-type	Min	Max	Skewness	Kurtosis
S&P 500	**0,00047**	**0,0135**	**–0,1277**	**0,0897**	**–0,82**	**17,15**
ETF Techno	**0,00073**	**0,0176**	**–0,1487**	**0,1109**	**–0,41**	**10,82**
Apple	**0,00096**	**0,0199**	**–0,1377**	**0,1132**	**–0,11**	**8,35**
Microsoft	**0,00077**	**0,0192**	**–0,1595**	**0,1329**	**–0,28**	**10,51**
Obligations (TLT)	**–0,00036**	**0,0113**	**–0,0690**	**0,0725**	**0,11**	**7,04**
ETF S&P 500 (SPY)	**0,00047**	**0,0133**	**–0,1159**	**0,0867**	**–0,81**	**15,16**
Euro/Dollar	**–0,00006**	**0,0047**	**–0,0281**	**0,0182**	**–0,15**	**4,98**
Bitcoin	**0,00141**	**0,0341**	**–0,4647**	**0,1718**	**–1,40**	**24,45**
Futur Pétrole	**0,00061**	**0,0337**	**–0,2822**	**0,3196**	**0,04**	**26,73**
Or (Gold)	**0,00043**	**0,0103**	**–0,0511**	**0,0578**	**–0,32**	**6,59**

Kurtosis calculée avec moments::kurtosis (non centrée : la valeur de référence d'une loi normale est 3). Skewness = coefficient d'asymétrie.

Plusieurs enseignements se dégagent :

- **Moyennes.** Elles sont très faibles (de l'ordre de 0,04 % à 0,14 % par jour pour les actifs à moyenne positive) devant les écarts-types, de sorte que le « signal » moyen est noyé dans le bruit. Deux actifs ont une moyenne négative : le TLT (–0,036 % par jour), dont le prix a fortement reculé avec la hausse des taux en 2022-2023, et l'euro contre dollar (–0,006 %), dans un contexte d'appréciation du dollar. Le Bitcoin affiche la moyenne la plus élevée

(0,14 %).

- **Risque (écart-type).** Par ordre croissant : euro/dollar (0,47 %), or (1,03 %), TLT (1,13 %), SPY et S&P 500 (≈ 1,33-1,35 %), ETF Techno (1,76 %), Microsoft (1,92 %), Apple (1,99 %), pétrole (3,37 %) et Bitcoin (3,41 %). À titre d'illustration, 1,35 % par jour correspond à une volatilité annualisée d'environ 21 % pour le S&P 500 (1,35 % × √252). On observe l'effet de diversification : l'indice est moins volatil que ses composantes (Apple, Microsoft) et que l'ETF sectoriel technologique.
- **Extrêmes.** Les valeurs minimales et maximales sont très éloignées de la moyenne : –0,465 pour le Bitcoin (soit environ –37 % en variation de prix), –0,28 et +0,32 pour le pétrole, lors des turbulences de mars-avril 2020 (voir figure 1b).
- **Asymétrie.** Elle est négative pour la majorité des actifs (S&P 500 : –0,82 ; Bitcoin : –1,40), c'est-à-dire que les grosses pertes sont plus marquées que les gros gains. Elle est proche de zéro pour le pétrole (0,04) et légèrement positive pour le TLT (0,11).
- **Queues épaisses.** La kurtosis est supérieure à 3 pour tous les actifs (de 4,98 pour l'euro/dollar à 26,73 pour le pétrole) : les rentabilités sont **leptokurtiques**. Les événements extrêmes sont bien plus fréquents que ne le prévoirait une loi normale ; l'hypothèse de normalité des rentabilités est donc inadaptée.

1.d Analyse visuelle de la stationnarité

Les prix (colonnes de gauche des figures 1a et 1b) ne sont pas stationnaires. Ils suivent des tendances durables (S&P 500 de ≈ 3 260 à ≈ 5 880 points ; or de ≈ 1 500 à ≈ 2 600 $; Bitcoin de ≈ 7 000 à plus de 90 000 $; baisse continue du TLT à partir de 2020) et ne reviennent pas vers un niveau moyen fixe. L'euro/dollar n'a pas de tendance globale mais connaît de longues excursions (≈ 1,22 début 2021, ≈ 0,96 en 2022, ≈ 1,04 fin 2024). La moyenne dépend donc de la date, et l'amplitude des fluctuations croît avec le niveau du prix (très net pour le Bitcoin), ce qui invalide aussi la constance de la variance.

Les rentabilités (colonnes de droite) oscillent autour d'une moyenne quasi nulle et constante, sans tendance : elles sont compatibles avec la stationnarité en moyenne. La variance n'est en revanche pas parfaitement uniforme : on repère des **regroupements de volatilité** (*volatility clustering*), avec un épisode très turbulent en mars-avril 2020 pour tous les actifs, et des phases plus agitées en 2022 pour les actions, le change et les obligations. Ces regroupements traduisent une variance *conditionnelle* changeante, mais ne remettent pas en cause la stationnarité faible tant que la variance *inconditionnelle* reste finie et constante ; ils montrent en revanche que les rentabilités ne sont pas indépendantes et identiquement distribuées (point repris en section 3.b).

2. Données simulées

2.a Modèles ARMA simulés

Dix processus ARMA(p, q) stationnaires ont été simulés avec 500 observations chacun (graine aléatoire 42). Le processus s'écrit :

$$X_t = \phi_1 X_{t-1} + \ldots + \phi_p X_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_{t-q}, \text{ avec } \varepsilon_t \sim i.i.d.\ N(0, 1)$$

Un processus ARMA est **stationnaire** si les racines du polynôme autorégressif $\Phi(z) = 1 - \phi_1 z - \ldots - \phi_p z^p$ sont toutes de module strictement supérieur à 1 (hors du cercle unité), et **inversible** si c'est le cas pour les racines du polynôme moyenne mobile $\Theta(z) = 1 + \theta_1 z + \ldots + \theta_q z^q$. Le tableau 3 donne les paramètres retenus et les modules minimaux de ces racines : tous les modèles sont stationnaires et inversibles.

Tableau 3 — Paramètres des modèles et vérification de la stationnarité

Modèle	ϕ (partie AR)	θ (partie MA)	Module min. racines AR	Module min. racines MA	Stationnaire / inversible
AR(1)	0,6	—	1,667	—	Oui / Oui

AR(2)	**0,5 ; –0,3**	**—**	**1,826**	**—**	**Oui / Oui**
AR(3)	**0,4 ; –0,2 ; 0,1**	**—**	**2,121**	**—**	**Oui / Oui**
MA(1)	**—**	**0,7**	**—**	**1,429**	**Oui / Oui**
MA(2)	**—**	**0,6 ; 0,3**	**—**	**1,826**	**Oui / Oui**
MA(3)	**—**	**0,5 ; 0,3 ; 0,2**	**—**	**1,701**	**Oui / Oui**
ARMA(1,1)	**0,6**	**0,4**	**1,667**	**2,500**	**Oui / Oui**
ARMA(1,2)	**0,5**	**0,4 ; 0,2**	**2,000**	**2,236**	**Oui / Oui**
ARMA(2,1)	**0,5 ; –0,3**	**0,4**	**1,826**	**2,500**	**Oui / Oui**
ARMA(2,2)	**0,4 ; –0,2**	**0,3 ; 0,2**	**2,236**	**2,236**	**Oui / Oui**

Les paramètres sont identiques en R (arima.sim, ar = ϕ, ma = θ) et en Python (ArmaProcess, polynômes [1, $-\phi$] et [1, θ]) : les modèles sont les mêmes, seules les réalisations aléatoires diffèrent.

2.b Trajectoires simulées

La figure 2 présente les dix trajectoires. Toutes fluctuent autour de zéro avec une amplitude stable, sans tendance ni changement de niveau : c'est le comportement attendu de processus stationnaires. Des différences de « texture » apparaissent toutefois : les modèles avec un coefficient autorégressif positif élevé (AR(1), ARMA(1,1), ARMA(1,2)) présentent des oscillations plus lentes et plus amples, tandis que l'AR(2) et l'ARMA(2,1), dont le second coefficient est négatif, oscillent plus rapidement. Les modèles MA, à mémoire courte, se rapprochent davantage d'un bruit légèrement lissé.

Figure 2 — Trajectoires simulées (n = 500) des dix modèles ARMA. Source : calculs de l'auteur (R).

2.c Propriétés statistiques

Les tableaux 4 et 5 comparent les moments empiriques (calculés sur les réalisations simulées avec R) aux valeurs théoriques du modèle. Les moyennes théoriques sont nulles (pas de constante) ; les variances et autocorrélations théoriques sont calculées à partir des paramètres (avec $\sigma^2 = 1$).

Tableau 4 — Moyenne et variance : valeurs empiriques et théoriques

Modèle	Moyenne empirique	Statistique z (H_0 : moyenne = 0)	Variance empirique	Variance théorique
AR(1)	**–0,116**	**–1,04**	**1,351**	**1,563**
AR(2)	**–0,042**	**–0,76**	**1,389**	**1,290**
AR(3)	**–0,047**	**–0,74**	**1,032**	**1,164**
MA(1)	**0,046**	**0,61**	**1,599**	**1,490**
MA(2)	**0,047**	**0,56**	**1,493**	**1,450**
MA(3)	**–0,075**	**–0,84**	**1,509**	**1,380**
ARMA(1,1)	**–0,100**	**–0,64**	**2,488**	**2,562**
ARMA(1,2)	**–0,201**	**–1,40**	**2,200**	**2,373**
ARMA(2,1)	**0,014**	**0,18**	**2,145**	**1,893**
ARMA(2,2)	**–0,017**	**–0,20**	**1,435**	**1,574**

La statistique z = moyenne / √(variance de longue période / n) tient compte de l'autocorrélation ; $|z| < 1{,}96 \Rightarrow$ moyenne non significativement différente de 0 au seuil de 5 %.

Tableau 5 — Autocorrélations empiriques (valeurs théoriques entre parenthèses)

Modèle	Retard 1	Retard 2	Retard 3	Retard 4	Retard 5
AR(1)	**0,562 (0,600)**	**0,287 (0,360)**	**0,154 (0,216)**	**0,121 (0,130)**	**0,082 (0,078)**
AR(2)	**0,396 (0,385)**	**–0,083 (–0,108)**	**–0,152 (–0,169)**	**–0,074 (–0,052)**	**–0,034 (0,025)**
AR(3)	**0,272 (0,330)**	**–0,037 (–0,035)**	**0,065 (0,020)**	**0,007 (0,048)**	**–0,054 (0,012)**
MA(1)	**0,515 (0,470)**	**0,080 (0,000)**	**0,105 (0,000)**	**0,093 (0,000)**	**0,062 (0,000)**
MA(2)	**0,544 (0,538)**	**0,203 (0,207)**	**–0,010 (0,000)**	**–0,007 (0,000)**	**0,006 (0,000)**
MA(3)	**0,510 (0,514)**	**0,314 (0,290)**	**0,103 (0,145)**	**–0,008 (0,000)**	**–0,030 (0,000)**
ARMA(1,1)	**0,752 (0,756)**	**0,468 (0,454)**	**0,271 (0,272)**	**0,153 (0,163)**	**0,122 (0,098)**
ARMA(1,2)	**0,736 (0,744)**	**0,460 (0,456)**	**0,234 (0,228)**	**0,118 (0,114)**	**0,063 (0,057)**
ARMA(2,1)	**0,541 (0,547)**	**–0,056 (–0,026)**	**–0,194 (–0,177)**	**–0,054 (–0,081)**	**0,078 (0,013)**
ARMA(2,2)	**0,560 (0,566)**	**0,221 (0,154)**	**0,028 (–0,052)**	**–0,036 (–0,051)**	**–0,067 (–0,010)**

Bornes de confiance à 95 % du corrélogramme : $\pm 1{,}96/\sqrt{500} \approx \pm 0{,}088$.

- **Moyennes.** Les moyennes empiriques sont proches de 0 (de –0,20 à +0,05) et aucune n'est significativement différente de la valeur théorique ($|z| < 1{,}5$ dans tous les cas) : les écarts

observés relèvent de l'aléa d'échantillonnage, particulièrement marqué pour les processus persistants.

- **Variances.** Elles sont du même ordre que les variances théoriques. La variance dépasse 1 (variance du bruit) et augmente avec la persistance : elle est maximale pour l'ARMA(1,1) (≈ 2,5) et l'ARMA(1,2) (≈ 2,2-2,4). Les écarts (par exemple 1,35 contre 1,56 pour l'AR(1)) restent compatibles avec la variabilité d'échantillonnage d'une série autocorrélée de 500 points.
- **Autocorrélations.** Elles reproduisent bien les valeurs théoriques et les « signatures » de chaque famille : décroissance géométrique pour l'AR(1) (0,56 → 0,29 → 0,15 → …) ; alternance de signes et amortissement rapide pour l'AR(2), l'AR(3) et l'ARMA(2,1) ; **coupure nette après le retard q** pour les MA (l'autocorrélation théorique est nulle au-delà ; les valeurs empiriques de 0,06 à 0,10 aux retards 2 à 5 du MA(1) sont du bruit d'échantillonnage, à la limite de la bande ou légèrement au-dessus) ; décroissance progressive pour les ARMA(1,1) et ARMA(1,2).

2.d Corrélogrammes et stationnarité

La figure 3 présente les corrélogrammes (ACF, 20 retards) ; les lignes pointillées bleues délimitent la bande de confiance à 95 % $\pm 1{,}96/\sqrt{n}$.

Figure 3 — Fonctions d'autocorrélation des dix séries simulées. Source : calculs de l'auteur (R).

Lecture. Chaque corrélogramme s'annule rapidement : la mémoire est courte (moins de 5 à 10 retards) et aucune décroissance lente vers zéro, signe typique d'une racine unitaire, n'est visible. Combinée à la constance de la moyenne et de la variance dans le temps (figure 2) et au fait que toutes les racines AR sont hors du cercle unité (tableau 3), cette observation confirme que les dix séries sont **stationnaires**. Les corrélogrammes des MA se coupent après le retard q ; ceux des AR et ARMA décroissent de façon géométrique ou sinusoïdale amortie.

Deux précautions. (i) Certains pics dépassent légèrement la bande à des retards éloignés (par exemple aux retards 11 à 17 pour l'ARMA(1,1) et l'ARMA(1,2)) : la bande $\pm 1{,}96/\sqrt{n}$ est calibrée pour un bruit blanc et sous-estime l'incertitude d'estimation lorsque la série est persistante ; ces dépassements relèvent de l'aléa d'échantillonnage et non d'une non-stationnarité. (ii) L'ACF seule ne permet pas d'identifier l'ordre d'un AR ou d'un ARMA (il faudrait aussi l'autocorrélation partielle, PACF), ce qui dépasse le cadre de ce projet.

3. Comparaison

3.a Prix et rentabilités : que dit la stationnarité ?

L'examen graphique (section 1.d) est confirmé par deux tests formels de racine unitaire et de stationnarité, appliqués aux dix séries de prix puis aux dix séries de rentabilités : le test **ADF** (Augmented Dickey-Fuller, hypothèse nulle : présence d'une racine unitaire, donc non-stationnarité) et le test **KPSS** (hypothèse nulle : stationnarité). Utiliser les deux tests ensemble est utile car leurs hypothèses nulles sont opposées : un résultat cohérent des deux renforce la conclusion.

Tableau 6 — Résultats des tests de stationnarité (seuil de 5 %)

Séries (10 actifs)	ADF (p-value)	KPSS (p-value)	Conclusion
Prix	**entre 0,14 et 0,92 (> 0,05) : H_0 non rejetée**	**0,01 (< 0,05) : H_0 rejetée**	**Non stationnaires (les deux tests concordent)**
Rentabilités	**0,01 (< 0,05) : H_0 rejetée**	**0,10 (> 0,05) : H_0 non rejetée**	**Stationnaires (les deux tests concordent)**

Les p-values 0,01 et 0,10 sont les bornes d'affichage des tests ADF et KPSS du package tseries : la vraie p-value est inférieure à 0,01 (ADF sur les rentabilités, KPSS sur les prix) ou supérieure à 0,10 (KPSS sur les rentabilités).

Ce qu'il en ressort. Les **prix** sont non stationnaires : ils se comportent comme des processus intégrés d'ordre 1, proches d'une marche aléatoire avec dérive, dont la moyenne et la variance

dépendent du temps. Les **rentabilités**, obtenues par différenciation du logarithme des prix, sont stationnaires : la transformation « log puis différence première » supprime la tendance stochastique et stabilise le niveau de la série. C'est la raison pour laquelle on modélise les rentabilités plutôt que les prix : les outils de la famille ARMA (dont l'estimation et l'inférence supposent la stationnarité) ne s'appliquent qu'à ces dernières. Nuance importante : la stationnarité est ici acquise en moyenne, mais la volatilité varie dans le temps (section 1.d).

3.b Rentabilités réelles et séries simulées : y a-t-il une ressemblance ?
La figure 4 présente les corrélogrammes des rentabilités réelles, à comparer à ceux des séries simulées (figure 3). La bande de confiance à 95 % est ici plus étroite ($\pm 1{,}96/\sqrt{1\ 257} \approx \pm 0{,}055$) car les échantillons sont plus longs.

Ressemblances. Comme les séries simulées, les rentabilités oscillent autour d'une moyenne constante, sans tendance, et leur autocorrélation s'éteint rapidement : elles sont bien stationnaires en moyenne.

Différences. En dehors de ce point commun, la dynamique est très différente :

- **Absence de structure autorégressive ou moyenne mobile marquée.** Pour l'euro/dollar, le Bitcoin, le pétrole, l'or et le TLT, les autocorrélations sont très faibles ($|\varrho| \leq 0{,}13$ à tous les retards ; ϱ_1 compris entre –0,07 et +0,04) et la majorité des barres reste dans la bande de confiance, avec quelques dépassements marginaux (TLT au retard 2 : –0,12 ; pétrole aux retards 3 et 7 ; Bitcoin au retard 1 : –0,07) : ces rentabilités se comportent comme un **bruit blanc**. Pour les actions et indices américains, on relève des autocorrélations statistiquement significatives mais faibles : $\varrho_1 = -0{,}18$ (S&P 500, Microsoft), –0,17 (ETF Techno), –0,15 (SPY), –0,11 (Apple), puis des pics alternés aux retards 2 et 6 à 9 (jusqu'à $\approx 0{,}19$ en valeur absolue pour le S&P 500). À titre de comparaison, les autocorrélations d'ordre 1 des séries simulées valent entre 0,27 (AR(3)) et 0,75 (ARMA(1,1)) (tableau 5). Ces pics pourraient être en partie tirés par les rentabilités extrêmes de mars 2020, l'ACF empirique étant sensible aux valeurs aberrantes : cette hypothèse serait à vérifier en excluant 2020 ou par un test de Ljung-Box.
- **Aucune série simulée ne reproduit ces corrélogrammes.** Les modèles AR, MA et ARMA choisis génèrent des dépendances linéaires nettes (décroissance géométrique ou coupure au retard q) ; les rentabilités réelles sont, elles, proches d'un bruit blanc. La série simulée la plus proche en termes d'ordre de grandeur serait un processus à très faible persistance, mais aucune des dix spécifications ne l'est vraiment.
- **Volatilité et distribution.** Les séries simulées sont gaussiennes et de variance constante ; les rentabilités réelles présentent des regroupements de volatilité, une asymétrie négative et des queues épaisses (kurtosis de 5 à 27, tableau 2). Cette caractéristique ne peut pas être capturée par un ARMA à bruit blanc gaussien ; elle relève de modèles de variance conditionnelle de type ARCH/GARCH.

Figure 4 — Fonctions d'autocorrélation des rentabilités réelles (20 retards). Source : calculs de l'auteur (R).

Interprétation économique. La quasi-absence d'autocorrélation des rentabilités est cohérente avec l'hypothèse d'efficience des marchés sous sa forme faible : les rendements passés ne permettent pas de prévoir linéairement les rendements futurs, sans quoi des stratégies simples en tireraient profit. Il faut cependant rester prudent : l'absence de corrélation linéaire n'implique pas l'indépendance, et les regroupements de volatilité montrent que les **rentabilités au carré** (ou en valeur absolue) sont, elles, très probablement autocorrélées, ce qu'il conviendrait de vérifier sur leur corrélogramme.

Tableau 7 — Synthèse de la comparaison

Critère	Prix	Rentabilités réelles	Séries ARMA simulées

Tendance / moyenne	Tendance nette, moyenne non constante	Moyenne proche de 0 et constante	Moyenne nulle et constante
Variance	Dépend du niveau (ex. Bitcoin)	Globalement stable mais regroupée en épisodes de forte volatilité (2020, 2022)	Constante (bruit gaussien homoscédastique)
Corrélogramme	—	Quasi nul ; quelques pics faibles pour les actions US	Structuré : décroissance géométrique ou coupure au retard q
Tests ADF / KPSS	Non stationnaires	Stationnaires	Stationnaires (racines hors du cercle unité)
Distribution	—	Asymétrique, queues épaisses (kurtosis 5 à 27)	Gaussienne (kurtosis = 3)

4. Complément : automatisation de l'analyse en Python

Conformément au guide, l'ensemble de la démarche a été automatisé en Python, avec l'aide d'un assistant d'intelligence artificielle (Claude, d'Anthropic) pour la structuration du code, puis relu et testé. Le script fournit deux fonctions autonomes, chacune de la forme « paramètres → analyse complète ». Le code complet est donné en annexe B.

4.a Analyse automatisée des données réelles

La fonction *analyser_donnees_reelles(tickers, start, end, lags)* enchaîne, pour un dictionnaire {nom : ticker} quelconque, les étapes des sections 1 et 3 : (1) téléchargement des cours avec *yfinance* ; (2) calcul des rentabilités logarithmiques, en détectant et supprimant proprement les prix négatifs ou nuls (cas du pétrole, avec message d'avertissement) ; (3) tracé des prix et des rentabilités ; (4) tableau de statistiques descriptives (moyenne, écart-type, min, max, skewness, kurtosis en excès) ; (5) corrélogrammes des rentabilités ; (6) tests **ADF** et **KPSS** sur les prix et sur les rentabilités, ainsi que test de **Ljung-Box** (hypothèse nulle : absence d'autocorrélation jusqu'au retard 10) pour évaluer formellement l'hypothèse de bruit blanc. Elle retourne un dictionnaire contenant prix, rentabilités et tableaux de résultats.

4.b Analyse automatisée des données simulées

La fonction *analyser_donnees_simulees(modeles, n, seed, lags)* réalise les étapes de la section 2 : simulation de chaque modèle ARMA (polynômes AR et MA passés sous forme de listes), vérification de la stationnarité et de l'inversibilité, tracé des trajectoires, calcul des moyennes et variances empiriques accompagnées de la **variance théorique** et des autocorrélations empiriques et théoriques aux retards 1 à 3, tests ADF/KPSS et tracé des corrélogrammes. Un nouveau modèle se teste en ajoutant simplement une entrée au dictionnaire *modeles*.

4.c Cohérence des résultats entre R et Python

Les deux implémentations aboutissent aux mêmes conclusions ; les petits écarts numériques s'expliquent par trois différences techniques :

- **Prix ajustés.** Les versions récentes de *yfinance* renvoient par défaut des cours de clôture *ajustés* des dividendes, alors que *Cl()* de *quantmod* renvoie le cours non ajusté. L'effet est visible pour le TLT (fort rendement en dividendes) : premier cours ≈ 112 $ en Python contre ≈ 137 $ en R, et moyenne des rentabilités légèrement différente.
- **Définition de la kurtosis.** *moments::kurtosis* (R) est non centrée et vaut 3 pour une loi normale ; *pandas.kurt()* (Python) est un excès de kurtosis corrigé du biais (0 pour une loi normale). Par exemple, pour le S&P 500 : 17,15 – 3 = 14,15 (R) contre 14,21 (Python).
- **Générateurs aléatoires.** R et NumPy n'utilisent pas la même suite pseudo-aléatoire : même avec la graine 42, les réalisations simulées diffèrent. Les moments empiriques varient donc d'un langage à l'autre, tout en restant proches des valeurs théoriques (tableau 8).

Tableau 8 — Exemples de résultats obtenus en R et en Python

Indicateur	R	Python	Valeur théorique
Variance simulée — AR(1)	1,351	1,466	1,563
Variance simulée — ARMA(1,1)	2,488	2,550	2,562

Variance simulée — ARMA(2,2)	**1,435**	**1,520**	**1,574**
Écart-type des rentabilités — S&P 500	**0,0135**	**0,0135**	**—**
Moyenne des rentabilités — TLT	**–0,00036**	**–0,0003**	**—**
Kurtosis en excès — S&P 500	**14,15**	**14,21**	**—**

Les variances Python du tableau 8 sont celles du notebook (numpy.var, sans correction de degré de liberté) ; le script fourni utilise ddof = 1, ce qui modifie les résultats de moins de 0,2 %.

Conclusion

Ce projet met en évidence trois résultats principaux.

- **Les prix financiers sont non stationnaires, les rentabilités le sont.** Les tests ADF et KPSS confirment l'analyse visuelle pour les dix actifs. Passer des prix aux rentabilités logarithmiques est donc l'étape préalable à toute modélisation de type ARMA.
- **Les modèles ARMA simulés ont des propriétés conformes à la théorie.** Moyennes nulles, variances et autocorrélations proches des valeurs théoriques ; signatures caractéristiques des corrélogrammes (décroissance géométrique pour les AR, coupure nette pour les MA).
- **Les rentabilités réelles ne ressemblent pas à ces processus.** Elles sont proches d'un bruit blanc (efficience faible des marchés), mais s'en distinguent par des queues épaisses, une asymétrie négative et des regroupements de volatilité, que la famille ARMA à bruit gaussien ne peut pas représenter.

Limites et prolongements. L'étude porte sur une seule période de cinq ans marquée par une crise exceptionnelle (2020), dont les valeurs extrêmes influencent les statistiques et certains corrélogrammes. Deux prolongements naturels sont : (i) tester formellement l'absence d'autocorrélation (Ljung-Box, déjà intégré au script Python) et étudier l'ACF des rentabilités au carré ; (ii) modéliser la variance conditionnelle par un modèle ARCH/GARCH, plus adapté à ces données.

Annexe A — Code R

Code de l'analyse principale (sections 1 à 3). Les tests de stationnarité ADF et KPSS (package *tseries*, fonctions *adf.test* et *kpss.test*) ont été appliqués à chaque série de prix et de rentabilités, avec les résultats du tableau 6.

```
library(quantmod)
library(moments)

tickers <- c("^GSPC", "XLK", "AAPL", "MSFT", "TLT", "SPY", "EURUSD=X", "BTC-USD", "CL=F", "GC=F")
noms <- c("S&P 500", "ETF Techno", "Apple", "Microsoft", "Obligations (TLT)",
          "ETF S&P 500 (SPY)", "Euro/Dollar", "Bitcoin", "Futur Pétrole", "Or (Gold)")

# Téléchargement (prix de clôture)
data <- list()
for (i in seq_along(tickers)) {
  temp <- getSymbols(tickers[i], src = "yahoo", from = "2020-01-01", to = "2025-01-01", auto.assign =
FALSE)
  data[[noms[i]]] <- Cl(temp)
}

# Rentabilités logarithmiques
rentabilites <- list()
for (nom in names(data)) {
  prix <- na.omit(data[[nom]])
  rentabilites[[nom]] <- na.omit(diff(log(prix)))
}

# Graphiques prix / rentabilités
for (nom in names(data)) {
  plot(as.numeric(data[[nom]]), type = "l", col = "steelblue",
       main = paste(nom, "- Prix"), xlab = "Temps", ylab = "Prix")
  plot(as.numeric(rentabilites[[nom]]), type = "l", col = "darkorange",
       main = paste(nom, "- Rentabilités"), xlab = "Temps", ylab = "Rentabilité")
}

# Statistiques descriptives
stats <- data.frame(
  Actif = names(rentabilites),
```

```
  Moyenne = sapply(rentabilites, mean),
  Ecart_type = sapply(rentabilites, sd),
  Min = sapply(rentabilites, min),
  Max = sapply(rentabilites, max),
  Skewness = sapply(rentabilites, skewness),
  Kurtosis = sapply(rentabilites, kurtosis)
)
print(stats)

# ---- Données simulées ----
set.seed(42)
n <- 500
modeles <- list(
  "AR(1)" = list(ar = 0.6, ma = NULL),
  "AR(2)" = list(ar = c(0.5, -0.3), ma = NULL),
  "AR(3)" = list(ar = c(0.4, -0.2, 0.1), ma = NULL),
  "MA(1)" = list(ar = NULL, ma = 0.7),
  "MA(2)" = list(ar = NULL, ma = c(0.6, 0.3)),
  "MA(3)" = list(ar = NULL, ma = c(0.5, 0.3, 0.2)),
  "ARMA(1,1)" = list(ar = 0.6, ma = 0.4),
  "ARMA(1,2)" = list(ar = 0.5, ma = c(0.4, 0.2)),
  "ARMA(2,1)" = list(ar = c(0.5, -0.3), ma = 0.4),
  "ARMA(2,2)" = list(ar = c(0.4, -0.2), ma = c(0.3, 0.2))
)
series_simulees <- list()
for (nom in names(modeles)) {
  m <- modeles[[nom]]
  series_simulees[[nom]] <- arima.sim(model = list(ar = m$ar, ma = m$ma), n = n)
}

# Trajectoires
for (nom in names(series_simulees)) {
  plot(series_simulees[[nom]], type = "l", col = "darkcyan",
       main = nom, xlab = "Temps", ylab = "Valeur")
}

# Moyennes, variances, autocorrélations (retards 1 à 5)
stats_simulees <- data.frame(
  Modele = names(series_simulees),
  Moyenne = sapply(series_simulees, mean),
  Variance = sapply(series_simulees, var)
)
autocorr_table <- data.frame(
  Modele = names(series_simulees),
  t(sapply(series_simulees, function(s) round(acf(s, lag.max = 5, plot = FALSE)$acf[2:6], 4)))
)

# Corrélogrammes (simulées puis réelles)
for (nom in names(series_simulees)) {
  acf(series_simulees[[nom]], main = paste("ACF -", nom), lag.max = 20)
}
for (nom in names(rentabilites)) {
  acf(rentabilites[[nom]], main = paste("ACF -", nom, "(réel)"), lag.max = 20)
}
```

Annexe B — Code Python (automatisation)

Script complet *automatisation_analyse.py*, fourni en fichier séparé avec ce rapport.

```
"""
Projet Économétrie des Séries Temporelles — Section 4 : Automatisation (Python)

Deux fonctions « clé en main » :
    - analyser_donnees_reelles(...)   : télécharge des séries Yahoo Finance, calcule les
      rentabilités logarithmiques, trace prix/rentabilités, calcule les statistiques
      descriptives, trace les corrélogrammes et teste la stationnarité (ADF, KPSS)
      ainsi que l'absence d'autocorrélation (Ljung-Box).
    - analyser_donnees_simulees(...)  : simule des modèles ARMA, trace les trajectoires,
      calcule moyennes/variances/autocorrélations (empiriques ET théoriques), trace les
      corrélogrammes et vérifie la stationnarité (racines du polynôme AR).

Dépendances : numpy, pandas, matplotlib, statsmodels (et yfinance pour le téléchargement).
"""

import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.arima_process import ArmaProcess
from statsmodels.tsa.stattools import acf, adfuller, kpss

warnings.filterwarnings("ignore", category=UserWarning)    # avertissements d'interpolation KPSS
warnings.filterwarnings("ignore", category=FutureWarning)  # changements d'API statsmodels (adfuller/kpss)

TICKERS_PAR_DEFAUT = {
    "S&P 500": "^GSPC",
    "ETF Techno": "XLK",
```

```
    "Apple": "AAPL",
    "Microsoft": "MSFT",
    "Obligations (TLT)": "TLT",
    "ETF S&P 500 (SPY)": "SPY",
    "Euro/Dollar": "EURUSD=X",
    "Bitcoin": "BTC-USD",
    "Futur Pétrole": "CL=F",
    "Or (Gold)": "GC=F",
}

MODELES_PAR_DEFAUT = {
    # (polynôme AR [1, -phi1, -phi2, ...], polynôme MA [1, theta1, theta2, ...])
    "AR(1)": ([1, -0.6], [1]),
    "AR(2)": ([1, -0.5, 0.3], [1]),
    "AR(3)": ([1, -0.4, 0.2, -0.1], [1]),
    "MA(1)": ([1], [1, 0.7]),
    "MA(2)": ([1], [1, 0.6, 0.3]),
    "MA(3)": ([1], [1, 0.5, 0.3, 0.2]),
    "ARMA(1,1)": ([1, -0.6], [1, 0.4]),
    "ARMA(1,2)": ([1, -0.5], [1, 0.4, 0.2]),
    "ARMA(2,1)": ([1, -0.5, 0.3], [1, 0.4]),
    "ARMA(2,2)": ([1, -0.4, 0.2], [1, 0.3, 0.2]),
}


# ---------------------------------------------------------------------------------------
# Outils communs
# ---------------------------------------------------------------------------------------
def _tests_stationnarite(serie, regression_adf="c"):
    """Retourne (p-value ADF, p-value KPSS). H0(ADF) = racine unitaire ; H0(KPSS) = stationnaire."""
    serie = pd.Series(serie).dropna().to_numpy()
    p_adf = adfuller(serie, regression=regression_adf, autolag="AIC")[1]
    p_kpss = kpss(serie, regression="c", nlags="auto")[1]
    return p_adf, p_kpss


def _grille(n_series, n_colonnes=2, hauteur_ligne=3.2):
    n_lignes = int(np.ceil(n_series / n_colonnes))
    fig, axes = plt.subplots(n_lignes, n_colonnes, figsize=(14, hauteur_ligne * n_lignes))
    return fig, np.atleast_1d(axes).flatten()


# ---------------------------------------------------------------------------------------
# 4.a — Données réelles
# ---------------------------------------------------------------------------------------
def analyser_donnees_reelles(tickers=None, start="2020-01-01", end="2025-01-01",
                             lags=20, prix=None, afficher=True):
    """
    Automatise l'analyse des données réelles.

    Paramètres
    ----------
    tickers : dict {nom: ticker Yahoo}. Défaut : les 10 actifs du projet.
    start, end : période de téléchargement.
    lags : nombre de retards pour les corrélogrammes.
    prix : dict {nom: pd.Series} optionnel. Si fourni, aucun téléchargement n'est effectué
           (utile hors-ligne ou pour tester la fonction).
    afficher : si True, appelle plt.show() pour chaque figure.

    Retour
    ------
    dict avec les clés : "prix", "rentabilites", "stats", "stationnarite", "ljung_box"
    """
    tickers = tickers or TICKERS_PAR_DEFAUT

    # 1. Téléchargement ------------------------------------------------------------
    if prix is None:
        import yfinance as yf  # import local : inutile si les prix sont fournis
        prix = {}
        for nom, ticker in tickers.items():
            df = yf.download(ticker, start=start, end=end, progress=False)
            prix[nom] = df["Close"].squeeze().dropna()
            print(f"{nom} ({ticker}) téléchargé : {len(prix[nom])} lignes")

    # 2. Rentabilités logarithmiques r_t = ln(P_t / P_{t-1}) -----------------------
    #    Le log n'est pas défini pour un prix <= 0 (ex. pétrole WTI le 20/04/2020) :
    #    ces observations sont signalées puis retirées.
    rentabilites = {}
    for nom, p in prix.items():
        p = p.astype(float)
        p_pos = p.where(p > 0)
        n_invalides = int(p_pos.isna().sum())
        r = np.log(p_pos / p_pos.shift(1)).dropna()
        if n_invalides:
            print(f"[Attention] {nom} : {n_invalides} prix <= 0 -> "
                  f"{len(p) - 1 - len(r)} rentabilités non définies supprimées")
        rentabilites[nom] = r

    # 3. Graphiques prix vs rentabilités -------------------------------------------
    fig, axes = plt.subplots(len(prix), 2, figsize=(14, 3 * len(prix)))
    for i, nom in enumerate(prix):
        axes[i, 0].plot(prix[nom], color="steelblue")
```

```
        axes[i, 0].set_title(f"{nom} - Prix")
        axes[i, 1].plot(rentabilites[nom], color="darkorange")
        axes[i, 1].set_title(f"{nom} - Rentabilités")
    plt.tight_layout()
    if afficher:
        plt.show()

    # 4. Statistiques descriptives ------------------------------------------------
    stats = pd.DataFrame({
        nom: {
            "Moyenne": r.mean(), "Écart-type": r.std(), "Min": r.min(), "Max": r.max(),
            "Skewness": r.skew(), "Kurtosis (excès)": r.kurt(),
        } for nom, r in rentabilites.items()
    }).T
    print(stats.round(4))

    # 5. Corrélogrammes des rentabilités ------------------------------------------
    fig, axes = _grille(len(rentabilites))
    for ax, (nom, r) in zip(axes, rentabilites.items()):
        plot_acf(r, ax=ax, lags=lags, title=f"ACF - {nom} (réel)")
    plt.tight_layout()
    if afficher:
        plt.show()

    # 6. Stationnarité (ADF + KPSS) et blancheur (Ljung-Box) ----------------------
    lignes, lb = [], {}
    for nom in prix:
        p_adf_p, p_kpss_p = _tests_stationnarite(prix[nom], regression_adf="ct")
        p_adf_r, p_kpss_r = _tests_stationnarite(rentabilites[nom], regression_adf="c")
        lignes.append({"Actif": nom,
                       "ADF prix": p_adf_p, "KPSS prix": p_kpss_p,
                       "ADF rentab.": p_adf_r, "KPSS rentab.": p_kpss_r})
        lb[nom] = acorr_ljungbox(rentabilites[nom], lags=[10], return_df=True)["lb_pvalue"].iloc[0]
    stationnarite = pd.DataFrame(lignes).set_index("Actif")
    ljung_box = pd.Series(lb, name="Ljung-Box (p-value, 10 retards)")
    print("\nTests de stationnarité (p-values) :")
    print(stationnarite.round(4))
    print("\nTest de blancheur des rentabilités :")
    print(ljung_box.round(4))

    return {"prix": prix, "rentabilites": rentabilites, "stats": stats,
            "stationnarite": stationnarite, "ljung_box": ljung_box}


# ---------------------------------------------------------------------------------------
# 4.b — Données simulées
# ---------------------------------------------------------------------------------------
def analyser_donnees_simulees(modeles=None, n=500, seed=42, lags=20, afficher=True):
    """
    Automatise l'analyse des données simulées (modèles ARMA).

    Retour
    ------
    (series_simulees, stats_simulees) : dict de séries et DataFrame récapitulatif
    (moyenne / variance empiriques, variance théorique, autocorrélations empiriques et
    théoriques aux retards 1-3, module minimal des racines AR, stationnarité, tests).
    """
    modeles = modeles or MODELES_PAR_DEFAUT
    np.random.seed(seed)
    series, lignes = {}, []

    for nom, (ar, ma) in modeles.items():
        proc = ArmaProcess(ar, ma)
        x = proc.generate_sample(nsample=n)
        series[nom] = x

        psi = proc.arma2ma(2000)                      # poids MA(inf) -> variance théorique
        rho_th = proc.acf(lags=4)[1:4]
        rho_emp = acf(x, nlags=3, fft=True)[1:4]
        racines_ar = np.abs(proc.arroots) if len(ar) > 1 else np.array([np.inf])
        p_adf, p_kpss = _tests_stationnarite(x)

        ligne = {"Modèle": nom, "Stationnaire": proc.isstationary, "Inversible": proc.isinvertible,
                 "Module min racines AR": racines_ar.min(),
                 "Moyenne": x.mean(), "Variance": x.var(ddof=1), "Variance théorique": (psi ** 2).sum(),
                 "ADF p-value": p_adf, "KPSS p-value": p_kpss}
        for k in range(3):
            ligne[f"ρ{k + 1} emp."] = rho_emp[k]
            ligne[f"ρ{k + 1} théo."] = rho_th[k]
        lignes.append(ligne)
        print(f"{nom} : stationnaire = {proc.isstationary}")

    stats_simulees = pd.DataFrame(lignes).set_index("Modèle")

    # Trajectoires
    fig, axes = _grille(len(series), hauteur_ligne=3.2)
    for ax, (nom, x) in zip(axes, series.items()):
        ax.plot(x, color="teal")
        ax.set_title(nom)
    plt.tight_layout()
    if afficher:
        plt.show()
```

```
    print(stats_simulees.round(4))

    # Corrélogrammes
    fig, axes = _grille(len(series))
    for ax, (nom, x) in zip(axes, series.items()):
        plot_acf(x, ax=ax, lags=lags, title=f"ACF - {nom}")
    plt.tight_layout()
    if afficher:
        plt.show()

    return series, stats_simulees


# ---------------------------------------------------------------------------------------
if __name__ == "__main__":
    # Simulées : fonctionne hors-ligne
    series_simulees, stats_simulees = analyser_donnees_simulees()

    # Réelles : nécessite une connexion internet (yfinance)
    resultats = analyser_donnees_reelles()
```